Multi-Agent Embodied Architecture muto-llm

1. Course Content

Understanding MutoRS's multi-agent embodied architecture, the muto-llm framework

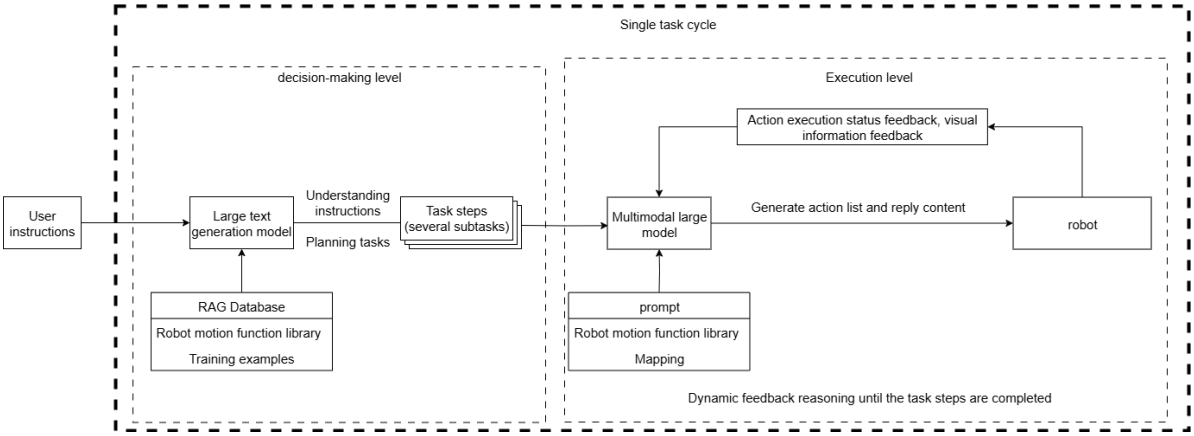
2. Introduction to muto-llm

- muto-llm is an embodied intelligence framework independently developed by Yabo Intelligence, adapted to general-purpose large AI models. It avoids the problems associated with VLA models and end-to-end models, such as requiring dedicated machines, local computing power, and long data collection and training times. By simply connecting to general-purpose AI models from cloud-based model providers, edge robots can achieve powerful embodied intelligence effects.
- muto-llm expands training examples through the RAG knowledge base, quickly enabling robots to adapt to different scenario tasks. It can be deployed on most small master control devices, allowing large-scale model inference to be placed in the cloud or on local servers.
- The core idea of muto-llm is to decouple different stages of a task through layered decoupling using multiple AI models. This is a comprehensive improvement upon the first-generation self-developed dual-model inference framework. Its main enhancements include:
- A smoother recording experience with inertial tail sound detection and adjustable tail sound detection duration.
- A more flexible decision-making mechanism, adding task routing functionality. The AI agent can autonomously choose between dual-model and single-model inference based on task difficulty, giving the AI greater autonomy and a better user experience.
- An easily understandable visual architecture process. muto-llm uses Dify to build a visual agent, allowing real-time observation of AI agent operation and data flow during debugging, enabling users to more intuitively understand the specific process of dual-model inference.
- A fully localized RAG knowledge base, eliminating concerns about data sensitivity.
- Seamless access to most global AI models. Through Dify's model vendor plugins, hundreds of global AI models can be quickly and seamlessly accessed (local models can also be accessed if needed).

- Traceable historical records. By querying Dify's backend access history, users can view all task instructions and the robot's execution steps.
- Personalized response voice. Users can customize random response voices. Built-in speech synthesis commands make it simple and easy to use.
- A comprehensive testing toolkit allows for quick verification of core functional modules.
- Simpler onboarding process, streamlining most operations and providing shortcut configurations.

3. Dual-Model Inference Architecture

- The design of **dual-model inference** and **dynamic feedback inference** provides stronger system robustness compared to a single-model architecture.



3.1 Advantages of Dual-Model Inference

3.1.1 Decoupling of Task Decision-Making and Action Transformation

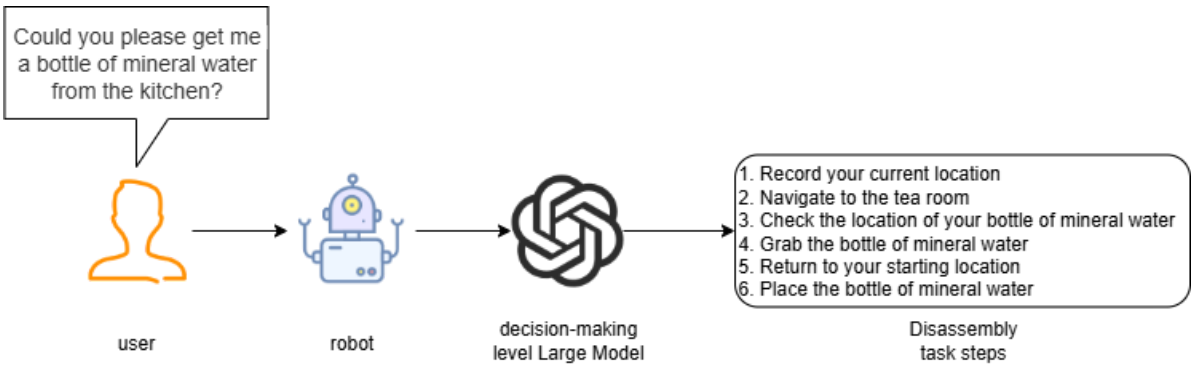
- The decision-making layer AI focuses on thinking, planning, and breaking down task instructions.
- The execution layer AI focuses on chat replies and converting task steps into JSON text.

3.1.2 Improving the Success Rate of Long-Flow Tasks

When using a single model for inference, it is necessary to simultaneously perform "natural language understanding + environmental perception + process decomposition + action function output," which is prone to **modal interference** (language ambiguity or excessively long instructions leading to misunderstandings). The dual-model approach, through phased processing, allows the decision-making layer to focus on task planning, and the execution layer to focus on action execution.

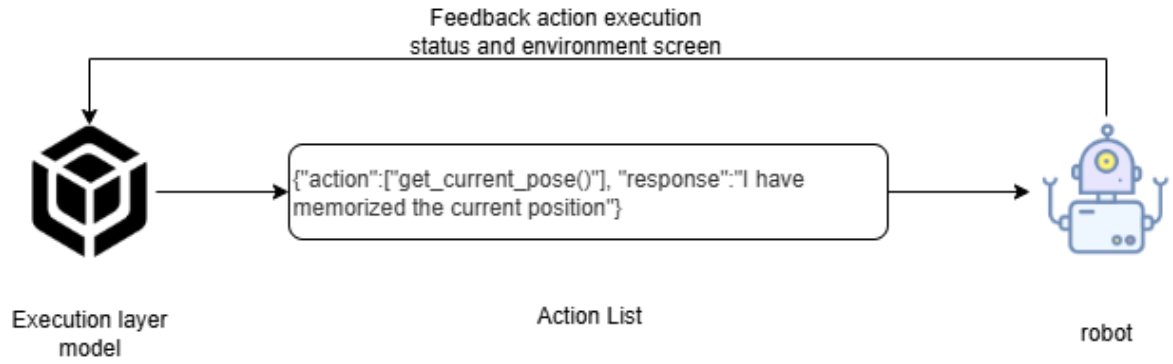
3.2 Decision-Making Layer AI

- The large decision-making layer model is mainly responsible for task planning. It can understand complex human instructions and decompose them into specific task steps. For example, when receiving the instruction "Can you get me a bottle of mineral water from the kitchen?", the large language model will break it down into several tasks, as shown in the diagram below.



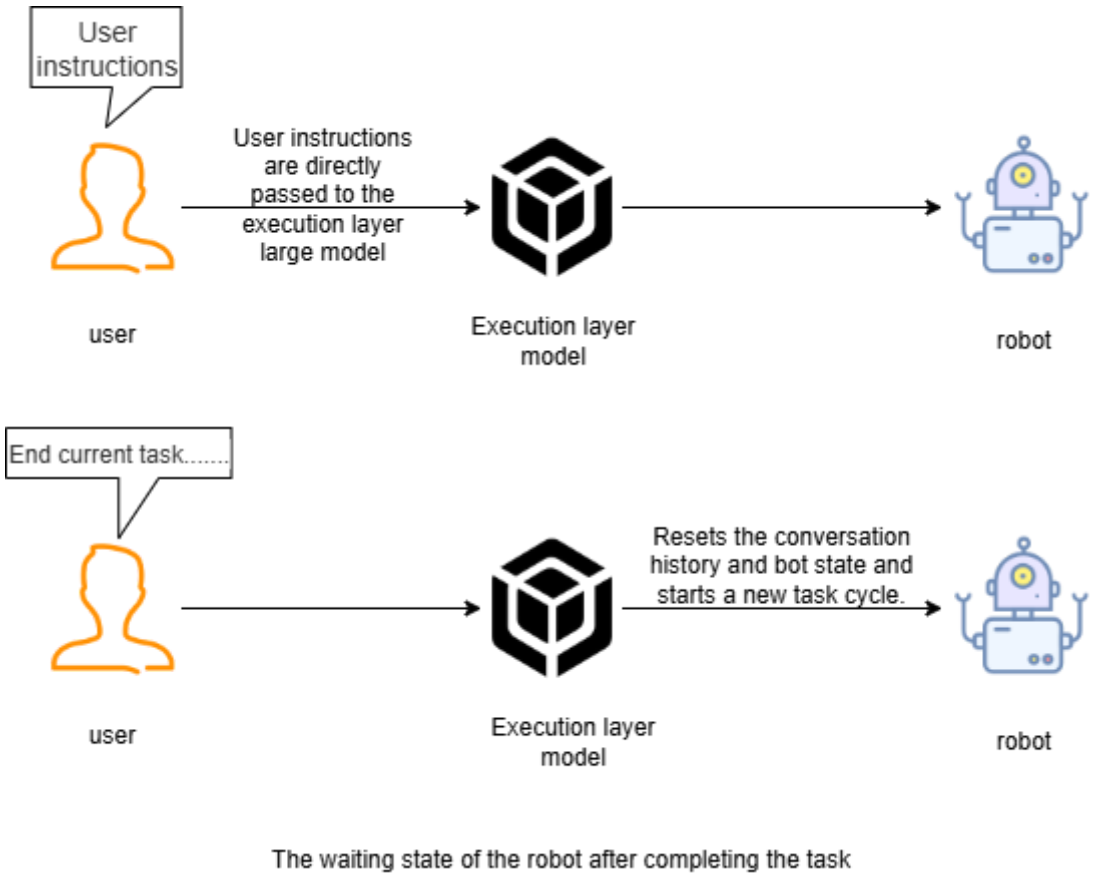
3.3 Execution Layer AI

- The execution layer model is mainly responsible for chat replies, converting task steps into JSON text, continuously receiving feedback (success/failure) and images from the robot's action results, and providing instructions for the next action based on the success or failure of the action execution.
- The execution layer model acts as a supervisor, continuously monitoring the robot's progress in executing task steps, and considering the feedback after the robot's actions and environmental information to determine the next action to be performed, until the task is successfully completed or terminated early due to special circumstances.



4. Task Cycle

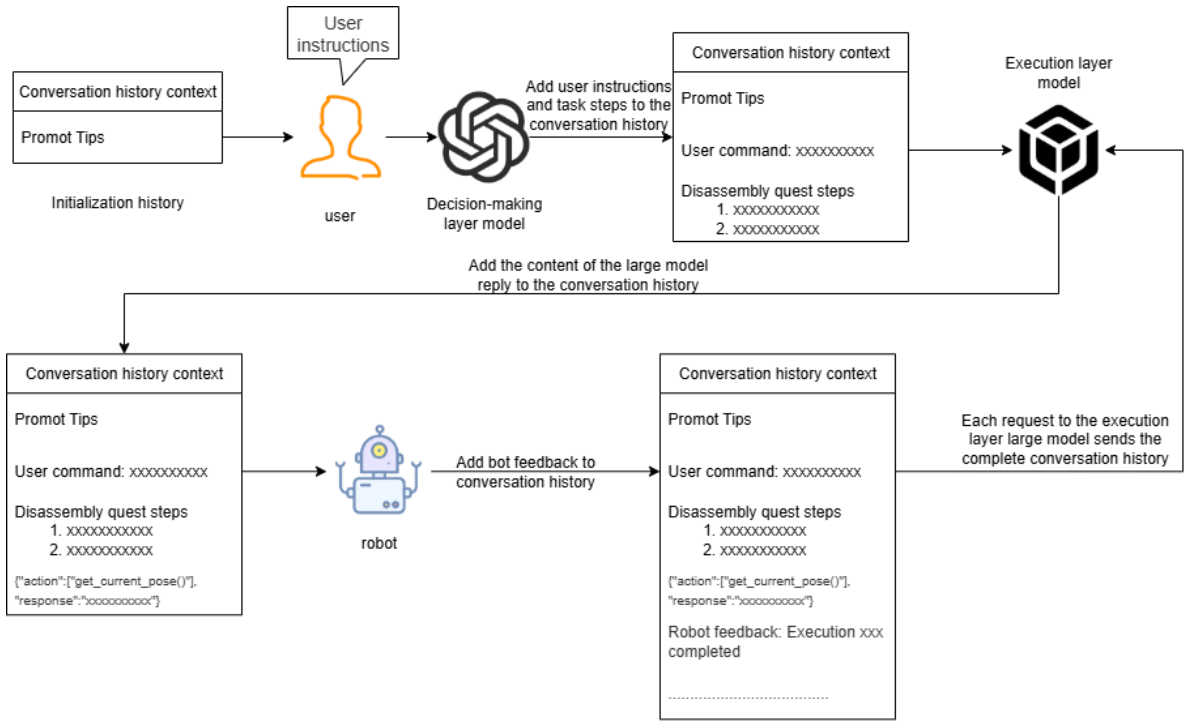
- After waking up the robot, a task cycle begins. All user commands and robot responses are stored in the AI model's historical context, essentially acting as the robot's temporary memory, allowing it to "remember" previous events.
- When the user requests to end the current task and allow the robot to rest, the robot resets the AI model's historical context, essentially causing the robot to "forget" previous events.



The waiting state of the robot after completing the task

5. Historical Context

- The robot's dialogue history is stored in DIfy's AI application variables, with a default maximum dialogue memory of 50 rounds.

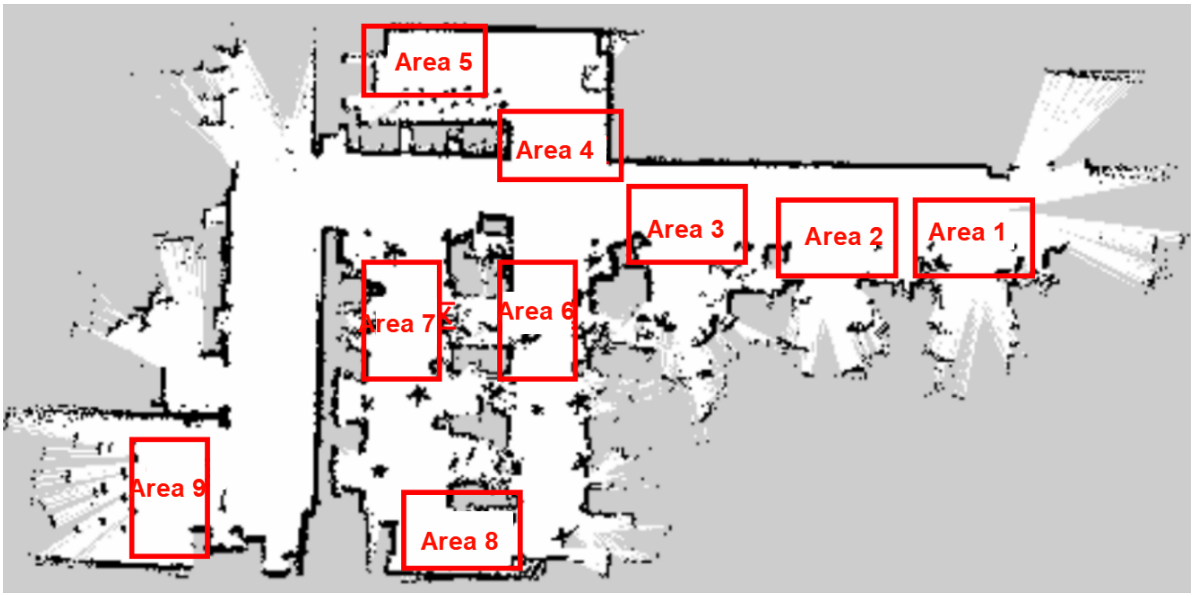


6. Map Mapping

- The robot uses a grid map for navigation. If the robot needs to understand its location in the real world, a mapping relationship needs to be established between the grid map and the

real-world environment. This relationship is called **map mapping**.

- Suppose we use a robot to generate a **grid map** using SLAM in a factory environment. The real factory has several manually divided areas, as shown in the image below:



We establish a one-to-one correspondence between these real-world areas and letter symbols:

A: "Area 1", B: "Area 2", C: "Area 3", D: "Area 4", E: "Area 5", F: "Area 6", G: "Area 7", H: "Area 8", I: "Area 9"

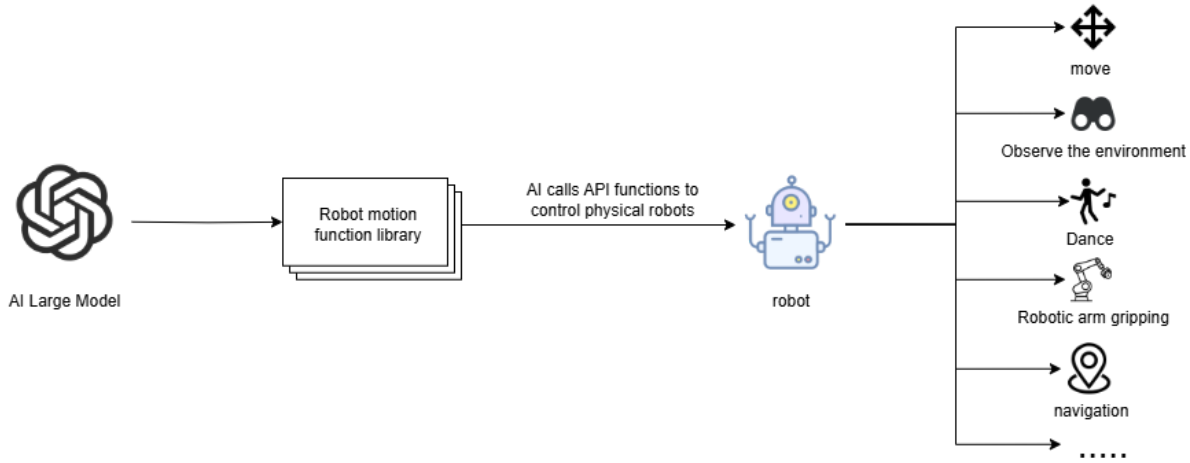
Then we write the map coordinates for the letter symbols in a YAML file, for example:

```yaml
A:
  name: 'Area 1'
  position:
    x: 4.4034953117370605
    y: 0.4879316985607147
  orientation:
    x: 0.0
    y: 0.0
    z: 0.701498621044694
    w: 0.7126708108744126
```

When we ask the robot to go to a certain actual area, we only need to convert the large model into the corresponding letter symbols, so that the robot can understand the location of the area in the real environment.

7. Action Function Library

- The API functions in the robot action function library are the bridge for the large model to control the robot to interact with the real world.
- These API functions define the minimum actions that the physical robot can perform in the physical world.
- The AI large model converts the actions to be performed into JSON text and sends it to the robot, which then parses the JSON text and executes the corresponding actions.



All minimum action functions and their corresponding functionalities are shown in the table below:

 # Muto Hexapod Capability Documentation

This document lists the core capability functions of the Muto robot.

1. Basic Movement Control

Supported parameter priority: `steps` > `distance_m` > `time_s` + `speed_ms`

- `forward(steps, distance_m, time_s, speed_ms)`: Move forward/forward/walk xx meters/xx seconds/xx meters/seconds
- `backward(steps, distance_m, time_s, speed_ms)`: Move backward/backward/walk xx meters/xx seconds/xx meters/seconds
- `shift_left(steps, distance_m, time_s, speed_ms)`: Move left/leftward/walk xx meters/xx seconds/xx meters/seconds
- `shift_right(steps, distance_m, time_s, speed_ms)`: Move right/rightward/walk xx meters/xx seconds/xx meters/seconds
- `rotate(direction, angle_deg, time_s, ...`angular_speed_deg_s)`: Rotate in place. Direction optional 'left'/'right'

`stop()`: Immediately stop all current movements

2. Preset Actions

`adjust_height(2)`: Restore default posture

`say_hello()`: Say hello

`wave_no()`: Wave to indicate "no"

`fear_retreat()`: Back away in fear

`curl_up()`: Curl up

`stretch()`: Stretch

`warm_up_squat()`: Warm up by squatting/doing push-ups

`spin_in_place()`: Spin in place (action performance)

`big_stride()`: Take a big stride forward

`point_left_front(hold_time, amplitude)`: Point left front leg/raise hand. Amplitude range 10-60, default is 60, hold_time default is 5 seconds

- `point_right_front(hold_time, amplitude)`: Right front leg points/raises hand. Amplitude range 10-60, default is 60, hold_time default is 5 seconds
- `head_move(level, direction)`: Head (body) movement
- `adjust_height(level)`: Adjusts the robot's height, optional parameters 1~3, default is 2. Use 1 to lower the height, use 3 to raise the height.

3. Perception and Interaction

- `robot_speak(text, volume)`: Voice broadcast (non-blocking). text is required.
- `have_a_look(user_query)`: Visual observation and analysis. user_query is required, used to guide observation needs, and should return corresponding information according to user needs. Note: If subsequent tasks require operations based on visual results (such as measuring the distance to an object or tracking an object), please explicitly request the return of the object's center coordinates (x, y) in the query, or the object's position coordinates (x1, x2, y1, y2) in the image. For simple recognition or detection, specific coordinate information is not required.
- Distance measurement requires the center coordinates (x, y).
- Tracking an object requires the top-left and bottom-right corner coordinates (x1, x2, y1, y2).
- `capture_image(save_path)`: Takes a photo.
- `get_depth_at_point(x, y)`: Gets the distance (in meters) at the specified coordinates (x, y) in the depth camera image. Must be used in conjunction with visual recognition results.
- `get_battery_info()`: Gets the battery level.
- `get_status()`: Gets the robot's full status (attitude, battery level, connection status, etc.).
- `start_lidar(wait_time_s, check_before_start)`: Starts the radar. `wait_time_s` defaults to 10.0, `check_before_start` defaults to True.
- `stop_lidar(force_kill, wait_time_s)`: Stops the radar. `force_kill` defaults to False, `wait_time_s` defaults to 3.0.
- `get_lidar_data(timeout_s, sample_count)`: Gets the raw radar data. `timeout_s` defaults to 10.0, `sample_count` defaults to 1.
- `get_lidar_360_data(timeout_s)`: Gets 360-degree radar data. timeout_s defaults to 5.0
- `get_lidar_range_at_angle(target_angle, angle_tolerance, timeout_s)`: Retrieves radar ranging data at the specified angle. `target_angle` is required; `angle_tolerance` defaults to 0.2; `timeout_s` defaults to 5.0.
- `check_lidar_status()`: Checks the radar status.

4. Navigation and Positioning

- `set_initial_pose(x, y, yaw)`: Sets the initial pose.
- `navigate_to_pose(x, y, yaw, timeout_s)`: Navigates to the coordinates.
- `save_current_position(position_name)`: Saves the current position.
- `navigate_to_saved_position(position_name)`: Navigates to the saved position.
- `list_saved_positions()`: Lists the saved positions.
- `get_saved_position(position_name)`: Retrieves the saved position information.
- `delete_saved_position(position_name)`: Deletes the saved position.

- `clear_all_positions(confirm)`: Clears all saved position information.

5. Flow Control

`wait_for_async_completion(request_id, timeout)`: Waits for the asynchronous task with the specified request_id to complete. request_id is optional (defaults to the current task), and timeout defaults to 30 seconds. Used to ensure that all background asynchronous actions (such as voice or mobile) have completed.

6. Action Learning

- `prepare_learning_posture()`: Enters a ready-to-perform posture.
- `start_continuous_action_learning(sequence_name="<action name>")`: Starts continuous action learning (automatic sampling and recording).
- `play_action_sequence(filename="<action name>")`: Plays the action sequence.
- `play_continuous_action_sequence(filename="<action name>", playback_speed=1.0)`: Plays the continuous action sequence.
- `list_action_files()`: Lists saved action files.
- `delete_action_file(filename)`: Deletes an action file.

7. Autopilot and Line Following (or Tracking Objects of Color xx)

- `direct_line_follow(hsv_threshold, timeout_s)`: Starts autopilot line following. (Start tracking objects of color xxx)
- hsv_threshold: Must be a list of 6 HSV values `[h_min, s_min, v_min, h_max, s_max, v_max]`.
- Note: If your task is "follow the red line", please refer to the [Common Color HSV Reference Table] below and convert "red" to the corresponding HSV array `[0, 120, 70, 10, 255, 255]` before entering the parameter. Do not directly enter the string "red".
- timeout_s: Optional (default is no timeout)

Common Color HSV Reference Table

- Red: `[0, 120, 184, 65, 253, 255]`
- Yellow: `[20, 100, 100, 40, 255, 255]`
- Green: `[40, 100, 100, 85, 255, 255]`
- Blue: `[55, 187, 91, 112, 253, 255]`
- Purple: `[130, 100, 100, 160, 255, 255]`
- Orange: `[10, 100, 100, 20, 255, 255]
- Cyan: [85, 100, 100, 100, 255, 255]
- White: [0, 0, 200, 180, 30, 255]
- Black: [0, 0, 0, 180, 255, 50]

8. KCF Target Tracking

- `set_kcf_target(x1, y1, x2, y2)`: Sets and starts KCF tracking. x1, y1 are the coordinates of the top-left corner, and x2, y2 are the coordinates of the bottom-right corner.
- `stop_kcf_tracking()`: Stops KCF tracking.